

Cross-Dataset Pre-Impact Fall Detection on a Low-Cost Microcontroller: Generalisation, Domain Adaptation and Verified TinyML Deployment

Abstract—Wearable fall detection is routinely reported with within dataset accuracy above 99%, yet that figure says little about how a device behaves once it leaves the laboratory it was trained in. This paper tests that claim directly. We train a 7,947-parameter separable convolutional neural network for pre impact fall detection and evaluate it under a strict leave one dataset out protocol across four public datasets, SisFall, KFall, FallAllID and UMAFall. The resulting 22.5 KB INT8 model is deployed on an ESP32 microcontroller costing under BDT 500. Three results follow. First, binary F1 falls from 0.907 within dataset to between 0.344 and 0.835 on an unseen dataset, with the decline strongly asymmetric even between datasets that appear equivalent on paper. Second, of three domain adaptation methods applied in increasing order of on device cost, only the cheapest, per window instance normalization, improves transfer, while CORAL and adversarial adaptation both reduce performance. Third, where the normalization is placed determines whether the model survives quantization at all. As a network layer, it costs 30.95 points of macro F1 under INT8, while moving it into the preprocessing pipeline avoids this loss. Scored per trial on KFall in the same way as the benchmark literature, the model reaches 0.985 sensitivity and 0.991 specificity at a mean lead time of 318 ms. Finally, the paper argues that window level specificity, the metric commonly reported in this literature, is close to meaningless for a detector that makes 7,200 decisions per hour, and reports false alarms per hour instead.

Index Terms—fall detection, pre-impact, TinyML, cross-dataset generalisation, domain adaptation, quantisation, wearable sensors, ESP32

I. INTRODUCTION

Falls are a major cause of injury related death among older adults, and the harm caused by a fall can be significantly increased when help arrives too late. A wearable detector that identifies a fall before the body reaches the ground can do more than simply call for assistance. It could trigger a protective mechanism or alert a caregiver while the fall is still happening. Pre impact detection is therefore both a more useful and a more challenging problem. Two important gaps remain in the existing literature.

Untested generalization. In dataset accuracy can approach 99%, but cross repository studies show substantial performance degradation when a model is evaluated outside the dataset used for training [1], [2]. Most studies do not test this form of generalization.

Unverified deployment. Most reported inference experiments are conducted on desktop GPUs. Studies that combine leave one dataset out evaluation with actual measurements on a microcontroller remain uncommon.

This paper addresses both gaps and reports the results transparently, including cases where performance is lower than what existing literature might suggest. The main contributions are as follows:

1. **Leave one dataset out evaluation across four public datasets** using a domain adaptation ladder arranged according to on device cost. The results show that only the least expensive method improves cross dataset transfer (Section V B).
2. **Pre impact detection validated using video grounded temporal labels**, with results reported at the trial level for comparison with previous studies and at the window level using false alarms per hour. The latter provides a more practical measure of whether a detector is suitable for wearable use (Section V C).
3. **Deployment evaluation on an ESP32**, including a quantization finding that merits particular attention. The position of instance normalization relative to the quantization boundary determines whether the INT8 model remains functional (Section V F).

All code, frozen preprocessing constants, the exported model, and fold level result files are publicly available at github.com/arifshekhk8/preimpact-fall-detection-tinyml.

II. RELATED WORK

Datasets and annotation. SisFall [3] records data from 38 subjects at 200 Hz, but it provides only trial level labels. Musci et al. [4] later introduced per sample three class annotations based on expert review of sensor traces without video support. This approach has been criticized by subsequent studies because of its subjective nature. KFall [5] differs in this respect because its fall onset and impact frames are identified from synchronized video. This makes KFall a more reliable source for pre impact detection research and the primary dataset used in this study. FallAllID [6] and UMAFall [7] provide additional diversity through different hardware configurations and sensor placements.

Pre impact detection. Using KFall, Yu et al. [5] evaluated threshold based, SVM, and ConvLSTM approaches. Their ConvLSTM model achieved 99.32% sensitivity and 99.01% specificity, with a lead time of 403 ± 163 ms. TinyFallNet [8] was developed for resource constrained devices and reported 86.67% sensitivity and 97.97% specificity with a lead time of 477.7 ± 5.8 ms. PreFallKD [9] achieved a lead time of 551.3 ms using knowledge distillation.

Cross dataset evaluation. Silva et al. [1] evaluated 13 algorithms across 11 datasets and found that all of them experienced performance degradation under cross dataset conditions. The existence of this generalization problem is therefore well established. The more practical question is which mitigation techniques can be used on a microcontroller at an acceptable cost. This study addresses that question and finds a result that differs from the expected ordering of methods.

Microcontroller deployment. Torti et al. [10] demonstrated a recurrent neural network on a wearable platform using the same type of annotations discussed above. However, studies that combine cross dataset evaluation with measured microcontroller performance remain limited.

III. DATA AND PREPROCESSING

A. We use four public datasets in this study. Before model development, we conducted an audit of the available data and annotations. This audit identified several limitations that directly affect what can be concluded from the experiments. We report these limitations explicitly rather than treating them as issues discovered later in the review process.

1. SisFall Enhanced annotations could not be reliably recovered. The available public mirror provides shuffled, pre windowed tensors without subject identifiers. We attempted to realign these data with the original recordings using correlation-based matching. The best recovery rate was 22.5%, compared with a time reversed null control, and the rate decreased further when stronger low pass filtering was applied. We therefore treat 22.5% as an empirical upper bound on recoverability rather than evidence of successful reconstruction. As a result, the video grounded temporal validation reported in Contribution 2 relies on KFall alone.

2. The available SisFall mirror contains only 25 of the original 38 subjects. In particular, 13 of the 15 older participants are absent. This is important because the older participant group is one of the main reasons SisFall is valuable for evaluating fall detection in the target population. Consequently, results obtained from this mirror should not be interpreted as representing the complete SisFall cohort.

3. UMAFall has a waist sensor sampling rate of 20.1 Hz, which is below the 50 Hz sampling rate used in our experiments. Only the pocket mounted smartphone provides data at 200 Hz, but that stream does not include gyroscope measurements. Therefore, the UMAFall fold introduces not

only a domain and sensor placement shift, but also a change in sensing modality.

4. FallAID uses a different gravity axis and sign convention from SisFall and KFall. If this difference is left uncorrected, it can appear as genuine cross dataset domain shift even though it is simply a difference in sensor orientation and coordinate convention. We therefore correct the coordinate convention before evaluating cross dataset performance. Without this correction, the experiment would partly measure the effect of an inverted sensor rather than a true difference between data distributions.

We exclude UP Fall from the study because its accelerometers operate at approximately 18.4 Hz. Resampling these signals to our 50 Hz working rate would require interpolation that cannot recover the missing high frequency information and could therefore introduce artificial signal content. For this reason, UP Fall is not included in the cross-dataset experiments.

B. Frozen preprocessing

Preprocessing is defined in a single module that is shared by both the training scripts and the firmware generation process. A hash of this preprocessing configuration is included in every cached artifact to ensure that the same processing steps are used throughout the pipeline. The input channels consist of the waist or low back accelerometer and gyroscope measurements (ax, ay, az, gx, gy, gz). Signals are converted to g and deg/s using the scaling specified for each dataset. This conversion is verified by checking that the magnitude of the accelerometer signal at rest is 1.0 ± 0.05 g across all four corpora.

To make the coordinate systems consistent, all datasets are rotated to a common gravity convention using signed permutation matrices with determinant +1. This preserves the handedness of the gyroscope measurements. Before down sampling to 50 Hz, anti-alias filtering is applied to prevent high frequency components, particularly those associated with the impact transient, from being aliased into the passband. Each input window contains 100 samples across six channels, corresponding to 2.0 s of data, with a stride of 0.5 s.

All data splits are performed by subject or by dataset, never by window. With 75% overlap between consecutive windows, randomly splitting individual windows can place almost identical samples on both sides of the train and test boundary. Such leakage can produce accuracy above 99% while providing little evidence of how the model will perform on genuinely unseen data.

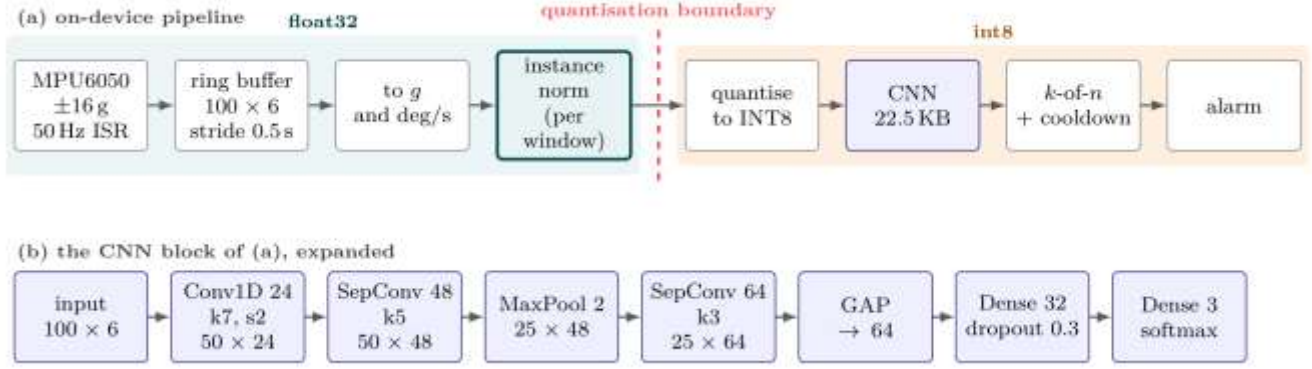


Fig. 1. The deployed system. (a) The on-device pipeline, shaded by numeric domain. Sampling is hardware-timer driven rather than a polled delay, because loop jitter accumulates and shifts window content away from the training distribution. Per-window instance normalisation is deliberately placed on the float side of the quantisation boundary: as a layer inside the graph it is mathematically identical but costs 30.95 points of macro-F1 under INT8 (Section V-F). (b) The network. Separable convolutions and global average pooling hold it to 7,947 parameters; batch normalisation and ReLU follow each convolution, and a flatten in place of the pooling would cost 51,200 parameters in a single layer.

IV. METHOD

The microcontroller memory budget is the main factor driving the network design. Figure 1 illustrates the architecture and the processing pipeline in which it is used. Each convolution is followed by batch normalization and ReLU activation, with dropout set to 0.3 before the output head. Separable convolutions and global average pooling are used instead of a flattening layer to keep the model compact, resulting in a total of 7,947 parameters. Using a flattening layer with the same feature widths would increase the parameter count to 51,200 in a single layer. The network uses a three class output head consisting of background, alert, and fall. For binary evaluation, the alert and fall classes are combined. The model is trained using Adam with cosine learning rate decay starting from 0.001. Training uses a batch size of 128 for a maximum of 100 epochs, with early stopping based on validation macro F1 and a patience of 15 epochs. Class weighting is used instead of oversampling. With a 75% window overlap, oversampling can introduce additional copies of highly similar windows and increase the risk of leakage across the data split.

A. Adaptation ladder

We evaluate three domain adaptation methods in order of increasing on device computational cost. The first is per window instance normalization, which requires only two passes over 100 samples and has negligible computational overhead on a 240 MHz processor. The second is CORAL [11], which aligns second order feature statistics between domains. The third is DANN [12], which uses adversarial adaptation through gradient reversal. None of these methods uses target domain labels at any stage.

V. RESULTS

A. Within-dataset performance is saturated

Table I supports the premise, with all learned models achieving macro F1 scores between 0.950 and 0.977. The proposed network achieves roughly two points less than the

best performing comparator, but reduces the number of parameters by approximately thirteen times compared with the 1D CNN.

TABLE I

Within-dataset baseline, SisFall, post-fall task. Comparators use 5-fold subject-grouped CV; the proposed model additionally uses full 25-fold LOSO.

Model	Params	Sens.	Spec.	Macro-F1
SMV threshold	2	0.763	0.934	0.484
SVM	—	0.958	0.997	0.977
Random Forest	—	0.952	0.997	0.975
1D-CNN	102,211	0.956	0.995	0.969
CNN-LSTM	47,811	0.954	0.996	0.972
Proposed	7,947	0.913	0.993	0.950
Proposed (LOSO)	7,947	0.882	0.993	0.953

Table II presents the same comparison for the task used in the final deployment. The overall ranking remains similar: the proposed model is approximately five macro F1 points behind the best comparator, but uses only 1/57 of the node count of the Random Forest model. The SVM threshold detector achieves a macro F1 of 0.326, highlighting the limitations of relying on a manually tuned rule for the pre impact detection task.

TABLE II

Within-dataset pre-impact baselines on KFall, three-class task, 5-fold subject-grouped CV.

Model	Params	Sens.	Spec.	Macro-F1
Random Forest	451,336	0.922	0.994	0.920
SVM	—	0.935	0.987	0.914
1D-CNN	102,211	0.936	0.985	0.914
CNN-LSTM	47,811	0.958	0.973	0.907
Proposed	7,947	0.936	0.959	0.874
SMV threshold	2	0.737	0.911	0.326

B. Cross-dataset generalisation

Table III presents the central result of the study. The proposed model achieves a within dataset binary F1 of 0.907, but its performance decreases substantially when evaluated on an unseen dataset. The decrease is also highly asymmetric. Transfer to KFall remains relatively strong, with an F1 of 0.835, whereas performance on FallAID drops to 0.344. This difference is notable because both datasets use waist mounted sensors with similar sampling rates. Figure 2 illustrates both the overall performance drop and the strong asymmetry between datasets.

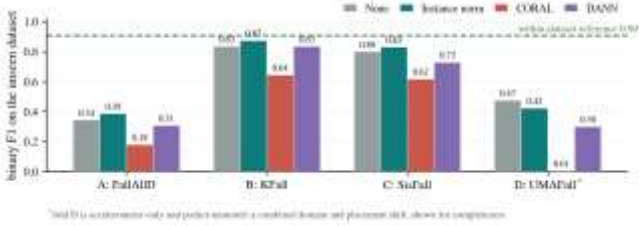


Fig. 2. Leave-one-dataset-out binary F1 (Table III), with the within-dataset reference of 0.907. Instance normalisation — the cheapest adaptation — is the best method on folds A–C; no method helps on fold D, which conflates domain, placement and modality shift.

TABLE III

Leave-one-dataset-out binary F1. Train on two corpora, test on a third; no fine-tuning and no target labels. Within-dataset reference: 0.907.

Fold	Test	None	Inst. n.	CORAL	DANN
------	------	------	----------	-------	------

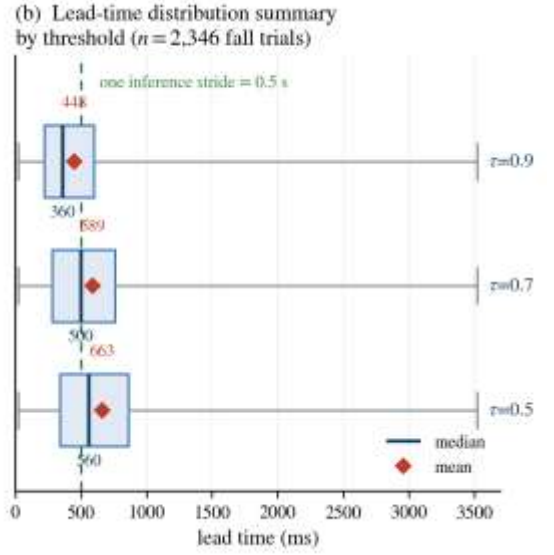
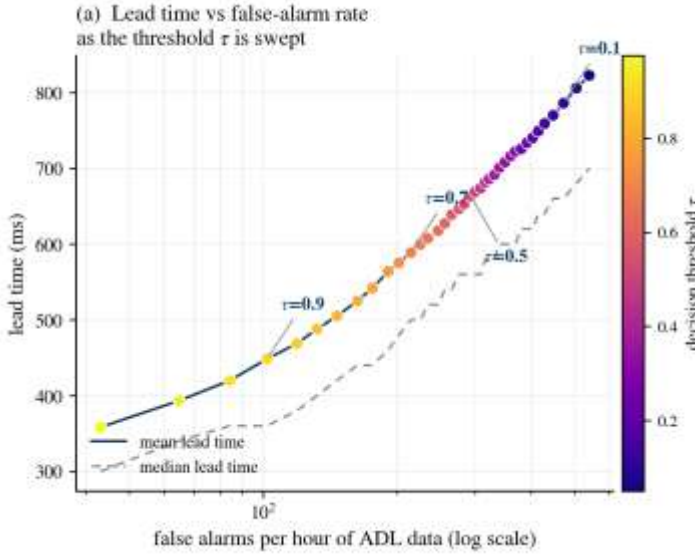


Fig. 3. The central operational trade-off on KFall, pooled over 32 LOSO folds. (a) Mean lead time against false alarms per hour of ADL data as the threshold is swept from 0.05 to 0.975. (b) Lead-time summary at three thresholds (box: min–Q1–median–Q3–max; diamond: mean); the dotted line marks one 0.5 s inference stride. Because the mean lead of 663 ms at threshold 0.5 is barely longer than a stride, any k-of-n debouncing consumes most of the lead time it is meant to protect, and the mean overstates the typical case, so the distribution is reported rather than the mean alone.

TABLE IV

Pre-impact performance on KFall, pooled over 32 LOSO folds, scored per trial as in [5]. Published rows are as reported in the cited works.

Method	Sens.	Spec.	Lead time (ms)
Threshold [5]	0.955	0.834	333 ± 160
SVM [5]	0.998	0.949	385 ± 159
ConvLSTM [5]	0.993	0.990	403 ± 163

A	FallAID	0.344	0.388	0.179	0.307
B	KFall	0.835	0.873	0.644	0.834
C	SisFall	0.799	0.830	0.617	0.728
D	UMAFall †	0.475	0.423	0.005	0.299

† Accelerometer-only, pocket placement: combined domain and placement shift.

The adaptation ladder inverts. Per window instance normalization is the least expensive method and the only one that can be implemented comfortably on the target hardware. It is also the only method that improves cross dataset transfer, increasing F1 by 0.031 to 0.044 across folds A to C. CORAL reduces performance on every fold, with F1 falling to 0.005 on UMAFall. DANN, which has the highest computational cost, performs worse than using no adaptation across all folds. These results suggest that the additional computational cost of more complex feature alignment methods does not provide a corresponding benefit in this setting. For practitioners limited to a single adaptation method, per window instance normalization provides the most practical choice because it improves transfer without adding significant computational cost.

C. Pre-impact detection

TinyFallNet [8]	0.867	0.980	478 ± 6
Ours ($\tau=0.5$, $k=3$)	0.985	0.991	318 ± 328
Ours ($\tau=0.7$, $k=3$)	0.978	0.995	342 ± 373

Table IV compares our model with published benchmarks using the same dataset and trial level evaluation protocol. Our model achieves sensitivity within one percentage point of the ConvLSTM benchmark and matches its specificity, although it has a shorter lead time. Importantly, this performance is

achieved with only 7,947 parameters. Our evaluation combines results from 32 leave one subject out folds rather than relying on a single held out split, providing a stricter assessment of generalization.

Window-level specificity is misleading. A detector that makes one decision every 0.5 s produces 7,200 decisions per hour. Therefore, even a window level specificity of 99% would result in 72 false alarms per hour, while achieving only one false alarm per hour would require a specificity of 0.99986. When evaluated at the window level, our model produces 293 false alarms per hour at a threshold of 0.5. Requiring k consecutive positive windows reduces the number of false alarms. For $k=3k=3$, the number of trial level false positives decreases from 538 to 26 among 2,729 ADL trials. However, each additional window introduces another 0.5 s stride of delay. At $k=1k=1$, the mean lead time is already 663 ms, only slightly longer than one stride. Figure 3 illustrates this tradeoff between early detection and false alarm rate. We argue that this balance is one of the main practical constraints for deploying pre impact fall detection, and that the decision stride itself plays an important role in this limitation.

D. Ablations

Figure 4 summarizes the ablation experiments. Window length has the largest effect on performance. A 2.0 s window achieves a binary F1 of 0.903, compared with 0.871 for a 1.5 s window and 0.804 for a 1.0 s window. Increasing the sampling rate from 50 Hz to 100 Hz improves F1 by only 0.006, while doubling both the computational cost and the input tensor size. These results support the use of 50 Hz based on measured performance rather than convention. Removing the gyroscope reduces F1 by 2.6 points, from 0.905 to 0.880, but saves 504 parameters and reduces the power required by the sensor. Across the three mounting positions in FallAllID, the neck placement performs best, with an F1 of 0.644, followed by the waist at 0.551 and the wrist at 0.476.

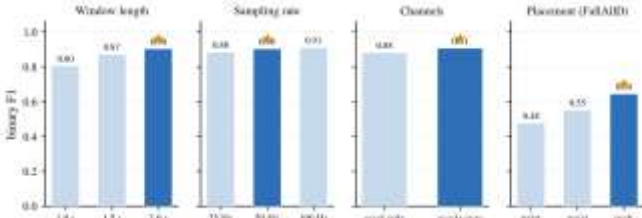


Fig. 4. Ablations (binary F1, 5-fold subject-grouped CV). The chosen configuration is shaded dark in each panel. Placement uses FallAllID rather than UMAFall, whose multi-position sensors all sample at 20 Hz, below the working rate.

E. Prototype system

The hardware platform is an ESP32 DevKit V1 with a 240 MHz dual core processor, 320 KB of usable RAM, and 4 MB of flash memory. It is connected to an MPU6050 IMU through I2C at 400 kHz, along with an active buzzer and a status LED. The system is powered by a 1000 mAh LiPo battery through a TP4056 charger. The total bill of materials is BDT 1,450, including BDT 450 for the microcontroller.

The IMU is configured to match the training setup exactly. It uses a ± 16 g measurement range with 2048 LSB/g sensitivity

and a ± 2000 dps range with 16.4 LSB/dps sensitivity. The built in digital low pass filter is enabled as the anti-aliasing stage, and the sample rate divider is set to 19, resulting in a sampling rate of $1000/(1+19)=501000/(1+19) = 50$ Hz. The ± 16 g range is necessary because impact events can exceed 8 g. A clipped acceleration peak would remove important information that the model relies on for detection.

A rule based two stage detector is also implemented using the same live sensor samples. It first identifies free fall when the acceleration magnitude drops below 0.65 g, followed by an impact above 2.50 g or rotation above 250 deg/s within 1.5 s. This provides an on-device implementation of the SMV threshold comparator reported in Table I, allowing the learned model and the rule-based detector to be evaluated using the same physical motion data.

F. Deployment and quantisation

TABLE V

Deployment on ESP32 DevKit V1 with MPU6050. The deployed model is trained on SisFall, KFall and FallAllID and evaluated on held-out subjects. Latency, peak RAM and battery life require the physical device and are deferred to future work (Section VII).

Metric	FP32	INT8
Parameters	7,947	
Model size (KB)	163.3	22.5
Tensor arena (KB)	—	≈ 8.0 (est.)
Macro-F1 (held out)	0.711	0.714
Sensitivity	0.922	0.922
FP32/INT8 agreement	0.994	

The location of normalization determines whether quantization succeeds.

Instance normalization is mathematically equivalent whether it is implemented as a network layer or applied during preprocessing. However, this equivalence does not hold in practice under full integer quantization. The mean and variance calculated for each window can produce intermediate tensors with a wide dynamic range that cannot be accurately represented using a single per tensor quantization scale:

Instance norm	FP32	INT8	Agreement
In graph	0.668	0.359	0.686
In pipeline	0.701	0.704	0.994

Moving the normalization operation across the boundary shown in Figure 1 changes a 30.95-point performance drop into no measurable loss. The firmware performs the same two pass calculation in floating point before quantization. This result is important because the failure is not immediately apparent: the FP32 model performs normally during validation, while the problem appears only after deployment of the INT8 model.

The deployed model does not use UMAFall for training because its available 200 Hz stream does not include gyroscope measurements. Treating the missing gyroscope channels as constant zero values during training could cause the network to learn that a non-functioning sensor is a normal input condition. Instead, the model is evaluated on held out subjects from the other three corpora, comprising 814 fall

trials and 1,136 ADL trials. It achieves a trial level sensitivity of 0.958 and specificity of 0.836, with a mean lead time of 539 ± 509 ms. Overall, 68% of detections occur before impact. Requiring two consecutive positive windows increases specificity to 0.975, but reduces the proportion of detections occurring before impact to 16%. This reproduces the tradeoff observed in Figure 3, but now under actual deployment rather than simulation.

These results are lower than those reported in Table IV because the Table IV model is trained and evaluated within KFall, whereas the deployed model must generalize across three heterogeneous datasets simultaneously.

VI. DISCUSSION AND LIMITATIONS

The cross-dataset performance gap is not uniform, and its asymmetry is more informative than its overall magnitude. FallAllD remains difficult to transfer to even though its sensor placement is nominally consistent with the training datasets. This suggests that differences in sensor hardware and recording procedures may have a greater influence on domain shift than sensor placement alone.

The study also has several limitations. First, the falls in the available datasets are simulated, mostly by younger participants, rather than being naturally occurring falls among older adults. Second, the available SisFall mirror is missing most of the older participant cohort. Third, annotations for all datasets except KFall were generated without video reference, which limits confidence in their temporal accuracy for pre impact evaluation. Fourth, the fold involving UMAFall combines differences in dataset, sensor placement, and sensing modality, making it difficult to isolate the individual contribution of each factor. Fifth, the false alarm rates reported in this study are based on laboratory ADL recordings rather than continuous free-living use. Finally, the hardware measurements corresponding to the cells in Table V have not yet been completed.

VII. CONCLUSION

A 7,947-parameter network compressed to a 22.5 KB INT8 model achieves pre impact detection performance comparable to a substantially larger published benchmark while running on a microcontroller that costs less than BDT 500. However, its strong within dataset performance does not transfer reliably to unseen datasets, and among the three adaptation methods evaluated, only the least expensive approach improves cross dataset performance. These results show that the main challenge for practical deployment is not simply achieving high accuracy. The more important issue is balancing detection lead time against false alarm rate. Reducing the inference stride therefore appears to be the most promising direction for future work, together with completing the remaining hardware measurements on the physical device.

REFERENCES

- [1] J. Silva, E. Casilari, and R. García-Bermúdez, "Cross-dataset evaluation of wearable fall detection systems using data from real falls and long-term monitoring of daily life," *Measurement*, vol. 234, p. 114843, 2024.
- [2] E. Casilari, J. A. Santoyo-Ramón, and J. M. Cano-García, "Analysis of public datasets for wearable fall detection systems," *Sensors*, vol. 17, no. 7, p. 1513, 2017.
- [3] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A fall and movement dataset," *Sensors*, vol. 17, no. 1, p. 198, 2017.
- [4] M. Musci, D. De Martini, N. Blago, T. Facchinetti, and M. Piastra, "Online fall detection using recurrent neural networks on smart wearable devices," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1276–1289, 2020.
- [5] X. Yu, J. Jang, and S. Xiong, "A large-scale open motion dataset (KFall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors," *Frontiers in Aging Neuroscience*, vol. 13, p. 692865, 2021.
- [6] M. Saleh, M. Abbas, and R. Le Bouquin Jeannès, "FallAllD: An open dataset of human falls and activities of daily living for classical and deep learning applications," *IEEE Sensors Journal*, vol. 21, no. 2, pp. 1849–1858, 2021.
- [7] E. Casilari, J. A. Santoyo-Ramón, and J. M. Cano-García, "UMAFall: A multisensor dataset for the research on automatic fall detection," *Procedia Computer Science*, vol. 110, pp. 32–39, 2017.
- [8] B. Kim, S.-W. Kim, and S. Lee, "TinyFallNet: A lightweight pre-impact fall detection model," *Sensors*, vol. 23, no. 20, p. 8459, 2023.
- [9] T.-H. Wu, C.-H. Chao, C.-Y. Lin, and Y.-C. Tseng, "PreFallKD: Pre-impact fall detection via CNN-ViT knowledge distillation," *arXiv preprint arXiv:2303.03634*, 2023.
- [10] E. Torti, A. Fontanella, M. Musci, N. Blago, D. Pau, F. Leporati, and M. Piastra, "Embedding recurrent neural networks in wearable systems for real-time fall detection," *Microprocessors and Microsystems*, vol. 71, p. 102895, 2019.
- [11] B. Sun, J. Feng, and K. Saenko, "Return of frustratingly easy domain adaptation," *Proc. AAAI Conference on Artificial Intelligence*, 2016.
- [12] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.